Cairo University - Faculty of Engineering

EECE Department

4th Year – Second Semester

Neural Networks ELC4028

Spring Semester of Academic Year: 2025 / 2026

Assignment 1

Name	ID	Sec
ايريني حسني ماهر رمزي	9220175	1
محمد خالد محمد شتا	9203230	3
محب ميخائيل حشمت ميخائيل	9220638	3

Submitted to

Dr. Mohsen Rashwan

Dr. Mohammed Abdelghany

Date of Submission: April 9, 2026

Contents

1 Introduction **3**

2 Part 1: Regression Problems **4**
2.1 Problem 1: Regression Analysis of Insured Persons . . . 4
2.1.1 Problem Description . . . 4
2.1.2 Hand Analysis . . . 4
2.1.3 Simulation Analysis . . . 9
2.2 Problem 2: Regression Analysis of Heating Oil Usage . . . 12
2.2.1 Problem Description . . . 12
2.2.2 Simulation Analysis . . . 12

3 Part 2: ReducedMNIST Classification **14**
3.1 Dataset . . . 14
3.2 Implementation Procedure and Workflow . . . 14
3.3 Feature Extraction . . . 14
3.4 Classifiers and Hyperparameters . . . 15
3.5 Results Summary . . . 15
3.6 Best Results and Confusion Matrices . . . 15
3.7 Observations . . . 16
3.7.1 Feature Performance . . . 16
3.7.2 Best Results . . . 17
3.7.3 Linear vs. Nonlinear SVM . . . 17
3.7.4 Computational Efficiency . . . 17
3.8 Conclusion . . . 17

4 Part 3: Ways to Fasten the Labelling Process **18**
4.1 Pipeline 1: K-means Bootstrapping + Human-in-the-Loop SVM . . . 18
4.1.1 Problem Statement and Setup . . . 18
4.1.2 Block Diagram (Exact Steps and Values) . . . 18
4.1.3 Evaluation and Time Estimate . . . 18
4.1.4 Practical Ground-Truth Convergence Check (5% Hold-Out) . . . 20
4.2 Pipeline 2: Active Learning + Self-Training SVM . . . 21
4.2.1 Block Diagram/Flow chart . . . 21
4.2.2 Simulation results . . . 22
4.3 Pipeline 3: Moondream Prototyping to Local CNN-Based Hybrid . . . 24
4.3.1 Pipeline 3 Stages (Final Local Workflow) . . . 24
4.3.2 Key Commands Used . . . 24
4.3.3 MATLAB Online Oracle Scoring (`check_accuracy.p`) . . . 25
4.3.4 Final Artifacts and Results . . . 27
4.3.5 Additional Pipeline 3 Baseline: Qwen2-VL (Practical 500 Set) . . . 27
4.4 Part 3 Cross-Pipeline Comparison . . . 28
4.5 Conclusion . . . 30

5 Appendix **31**
5.1 Using OpenCLIP . . . 31
5.1.1 Step 1: Benchmarking on the Practical Ground Truth Set . . . 31

List of Figures

1 Scatterplot of insured persons vs. years since 1987 with polynomial fits (original data) 9
2 R-squared values for the original data 10
3 Scatterplot of insured persons vs. years since 1987 with polynomial fits (cleaned data) 10
4 R-squared values for cleaned data 10
5 Best cleaned model providing a forecast for 1997 11
6 R-squared values for the original data 12
7 R-squared values for cleaned data 12
8 Best cleaned model and prediction for temperature $15°$F with insulation 5 inches . . 13
9 Confusion matrix for the best K-means result: K-means ($K = 16$) with HOG features. 16
10 Confusion matrix for the best SVM result: linear SVM with HOG features. 16
11 Pipeline 1 block diagram with exact run settings and observed values. 18
12 Flow chart for pipeline 2 with the exact steps and values 21

1 Introduction

This report documents the methodology and findings for Assignment 1, which explores a progression of techniques ranging from classical machine learning methods to the application of Large Language Models (LLMs). The assignment is divided into three parts:

- Part 1: Regression problems.
- Part 2: Classical machine learning modeling using ReducedMNIST.
- Part 3: Approaches to accelerate the labeling process using advanced techniques, including LLMs.

The full simulation, plots, and numerical computations are provided in the accompanying GitHub repository: ELC4028-Neural-Networks.

2 Part 1: Regression Problems

2.1 Problem 1: Regression Analysis of Insured Persons

2.1.1 Problem Description

The objective of this problem is to analyze historical data concerning the number of insured persons over a ten-year period (1987–1996) and fit an appropriate regression model.
The goal is to analyze the dataset for potential outliers, evaluate linear, quadratic, and cubic polynomial fits to determine the most suitable model, and ultimately predict the number of insured persons for the year 1997.

2.1.2 Hand Analysis

We will first analyze the **original dataset**, then remove the outlier and refit.

Step 1: Transform Year for Numerical Stability
The dataset provides the yearly count of insured persons (y), which is modeled against the number of years since 1987 ($x = \text{Year} - 1987$, such that $x \in \{0, 1, \ldots, 9\}$).

x	0	1	2	3	4	5	6	7	8	9
y	12400	10900	10000	1050	9500	8900	8000	7800	7600	7200

Step 2: Fit Models (Original Data)
We consider the following polynomial models:

$$\begin{aligned} \text{Linear:} \quad & y = a_0 + a_1 x \\ \text{Quadratic:} \quad & y = b_0 + b_1 x + b_2 x^2 \\ \text{Cubic:} \quad & y = c_0 + c_1 x + c_2 x^2 + c_3 x^3 \end{aligned}$$

Linear Regression Analysis
For the original dataset with $n = 10$ points, we perform a detailed linear regression analysis to fit the model $y = a_0 + a_1 x$. **Computing the Regression Coefficients:**

x_i	y_i	$(x_i - \bar{x})$	$(y_i - \bar{y})$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$	$(x_i - \bar{x})(y_i - \bar{y})$
0	12400	-4.5	4065	20.25	16524225	-18292.5
1	10900	-3.5	2565	12.25	6579225	-8977.5
2	10000	-2.5	1665	6.25	2772225	-4162.5
3	1050	-1.5	-7285	2.25	53070225	10927.5
4	9500	-0.5	1165	0.25	1357225	-582.5
5	8900	0.5	565	0.25	319225	282.5
6	8000	1.5	-335	2.25	112225	-502.5
7	7800	2.5	-535	6.25	286225	-1337.5
8	7600	3.5	-735	12.25	540225	-2572.5
9	7200	4.5	-1135	20.25	1288225	-5107.5
Sum				82.5	8.29×10^7	-30325

- **Mean:** $\bar{x} = \frac{0+1+2+\cdots+9}{10} = 4.5$
- **Mean:** $\bar{y} = \frac{\sum y_i}{10} = 8335$

Using the regression formulas:

$$a_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{-30325}{82.5} \approx -367.58$$

$$a_0 = \bar{y} - a_1\bar{x} = 8335 - (-367.58)(4.5) = 8335 + 1654.09 \approx 9989.09$$

Thus, the linear model for the original data is:

$$y \approx 9989.1 - 367.6x$$

Steps for Computing SSE and R^2:

1. Compute predicted values for each point using the fitted model:

$$\hat{y}_i = a_0 + a_1 x_i$$

2. Compute residuals (prediction errors):

$$e_i = y_i - \hat{y}_i$$

3. Square and sum the residuals to obtain the error sum of squares:

$$SSE = \sum e_i^2 = \sum (y_i - \hat{y}_i)^2$$

4. Compute the total variability around the mean:

$$SST = \sum (y_i - \bar{y})^2$$

5. Compute the coefficient of determination:

$$R^2 = 1 - \frac{SSE}{SST}$$

Substituting the values for the original dataset:

$$SSE_{\text{lin}} = \sum (y_i - \hat{y}_i)^2 \approx 7.17 \times 10^7$$

$$R^2_{\text{lin}} = 1 - \frac{SSE_{\text{lin}}}{SST} \approx 1 - \frac{7.17 \times 10^7}{8.29 \times 10^7} \approx 0.135$$

Observation: The linear model yields a low $R^2 \approx 0.135$, indicating a poor fit. The outlier at $(x = 3, y = 1050)$ severely distorts the linear trend.

Quadratic Model

A quadratic model takes the form $y = b_0 + b_1 x + b_2 x^2$.

Using the matrix method, define

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix}, \quad \beta = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

and solve the normal equations

$$(X^T X)\beta = X^T y.$$

For the original data ($n = 10$, $x = 0, \ldots, 9$):

$$X^T X = \begin{bmatrix} 10 & 45 & 285 \\ 45 & 285 & 2025 \\ 285 & 2025 & 15333 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 83350 \\ 344750 \\ 2174650 \end{bmatrix}.$$

Solving gives

$$\beta = \begin{bmatrix} 11627.73 \\ -1596.55 \\ 136.55 \end{bmatrix},$$

so the fitted quadratic model is

$$\hat{y} = 11627.73 - 1596.55x + 136.55x^2.$$

This model can bend around the outlier more than a line, and its fit metrics are:

$$SSE_{\text{quad}} \approx 6.19 \times 10^7$$

$$R^2_{\text{quad}} \approx 1 - \frac{6.19 \times 10^7}{8.29 \times 10^7} \approx 0.253$$

Observation: The quadratic model shows improvement over the linear model (R^2 increases from 0.135 to 0.253), but still provides a weak overall fit. The outlier prevents the quadratic from capturing the underlying trend across all points.

Cubic Model

A cubic model takes the form $y = c_0 + c_1 x + c_2 x^2 + c_3 x^3$.

Using the matrix method, define

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & x_n^3 \end{bmatrix}, \quad \beta = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix},$$

and solve

$$(X^T X)\beta = X^T y.$$

For the original data:

$$X^T X = \begin{bmatrix} 10 & 45 & 285 & 2025 \\ 45 & 285 & 2025 & 15333 \\ 285 & 2025 & 15333 & 120825 \\ 2025 & 15333 & 120825 & 978405 \end{bmatrix}, \quad X^T y = \begin{bmatrix} 83350 \\ 344750 \\ 2174650 \\ 15383150 \end{bmatrix}.$$

Solving gives

$$\beta = \begin{bmatrix} 13113.15 \\ -4313.93 \\ 932.31 \\ -58.95 \end{bmatrix},$$

so the fitted cubic model is

$$\hat{y} = 13113.15 - 4313.93x + 932.31x^2 - 58.95x^3.$$

With 4 parameters for 10 data points, the cubic model has more flexibility to bend around the outlier. Its fit metrics are:

$$SSE_{\text{cubic}} \approx 5.11 \times 10^7$$

$$R^2_{\text{cubic}} \approx 1 - \frac{5.11 \times 10^7}{8.29 \times 10^7} \approx 0.383$$

Observation: The cubic model exhibits the highest $R^2 \approx 0.383$, but this improvement is still largely driven by the model's increased flexibility to fit the extreme outlier rather than capturing a genuine cubic underlying trend.

Conclusion for Original Data

Although the cubic model has the highest R^2 value among the three models fitted to the original data, the overall poor performance across all models (even the best $R^2 \approx 0.383$ explains only about 38% of variance) demonstrates the severe impact of the outlier. The outlier at $(x = 3, y = 1050)$ fundamentally distorts the regression analysis, and no simple polynomial model can adequately fit both this extreme point and the general declining trend in the remaining data. This observation motivates the need for outlier detection and removal before proceeding with final model selection.

Step 3: Remove Outlier

Removing the obvious outlier $(x = 3, y = 1050)$ leaves $n = 9$ valid points.

Step 4: Fit Models Without Outlier

Linear Regression Analysis

For the remaining $n = 9$ data points, the x values are $\{0, 1, 2, 4, 5, 6, 7, 8, 9\}$. The detailed computations are shown below:

x_i	y_i	$(x_i - \bar{x})$	$(y_i - \bar{y})$	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$	$(x_i - \bar{x})(y_i - \bar{y})$
0	12400	-4.667	3255.556	21.778	10598641.98	-15192.59
1	10900	-3.667	1755.556	13.444	3082716.05	-6437.04
2	10000	-2.667	855.556	7.111	731975.31	-2281.48
4	9500	-0.667	355.556	0.444	126419.75	-237.04
5	8900	0.333	-244.444	0.111	59753.09	-81.48
6	8000	1.333	-1144.444	1.778	1309753.09	-1525.93
7	7800	2.333	-1344.444	5.444	1807530.86	-3137.04
8	7600	3.333	-1544.444	11.111	2385308.64	-5148.15
9	7200	4.333	-1944.444	18.778	3780864.20	-8425.93
Sum				80.000	2.39×10^7	-42466.67

- **Mean:** $\bar{x} = \frac{0+1+2+4+\cdots+9}{9} = 4.667$
- **Mean:** $\bar{y} = \frac{\sum y_i}{9} = 9144.44$

Using the regression formulas:

$$a_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{-42466.67}{80} \approx -530.83$$

$$a_0 = \bar{y} - a_1\bar{x} = 9144.44 - (-530.83)(4.667) = 9144.44 + 2477.23 \approx 11621.67$$

Thus, the best-fit linear equation is:

$$y \approx 11621.67 - 530.83x$$

Substituting the cleaned-data values into the previously defined SSE and R^2 equations:

$$SSE_{\text{lin}} = \sum(y_i - \hat{y}_i)^2 \approx 1.34 \times 10^6$$
$$R^2_{\text{lin}} = 1 - \frac{SSE_{\text{lin}}}{SST} \approx 1 - \frac{1.34 \times 10^6}{2.39 \times 10^7} \approx 0.944$$

The linear model already provides an excellent fit with $R^2 \approx 0.944$.

Quadratic Model

A quadratic model takes the form $y = b_0 + b_1x + b_2x^2$.

Using the matrix method defined previously, for the cleaned data:

$$X^TX = \begin{bmatrix} 9 & 42 & 276 \\ 42 & 276 & 1998 \\ 276 & 1998 & 15252 \end{bmatrix}, \quad X^Ty = \begin{bmatrix} 82300 \\ 341600 \\ 2165200 \end{bmatrix}.$$

Solving gives

$$\beta = \begin{bmatrix} 12024.17 \\ -863.13 \\ 37.44 \end{bmatrix},$$

so the fitted quadratic model is

$$\hat{y} = 12024.17 - 863.13x + 37.44x^2.$$

Its fit metrics are:

$$SSE_{\text{quad}} \approx 6.57 \times 10^5$$
$$R^2_{\text{quad}} \approx 1 - \frac{6.57 \times 10^5}{2.39 \times 10^7} \approx 0.972$$

Observation: The quadratic model improves the fit over linear (R^2 increases from 0.944 to 0.972), but adds one extra parameter.

Cubic Model

A cubic model takes the form $y = c_0 + c_1x + c_2x^2 + c_3x^3$.

Using the matrix method defined previously, for the cleaned data:

For the cleaned data:

$$X^TX = \begin{bmatrix} 9 & 42 & 276 & 1998 \\ 42 & 276 & 1998 & 15252 \\ 276 & 1998 & 15252 & 120582 \\ 1998 & 15252 & 120582 & 977676 \end{bmatrix}, \quad X^Ty = \begin{bmatrix} 82300 \\ 341600 \\ 2165200 \\ 15354800 \end{bmatrix}.$$

Solving gives

$$\beta = \begin{bmatrix} 12204.42 \\ -1221.74 \\ 140.17 \\ -7.46 \end{bmatrix},$$

so the fitted cubic model is

$$\hat{y} = 12204.42 - 1221.74x + 140.17x^2 - 7.46x^3.$$

Its fit metrics are:

$$SSE_{\text{cubic}} \approx 5.09 \times 10^5$$

$$R^2_{\text{cubic}} \approx 1 - \frac{5.09 \times 10^5}{2.39 \times 10^7} \approx 0.979$$

Observation: The cubic model gives the highest R^2, but the improvement from quadratic ($0.972 \to 0.979$) is modest relative to the extra complexity.

Conclusion for Cleaned Data

All three models fit the cleaned dataset well. However, following the same model-selection principle used in Step 2 (favor simpler models unless improvement is substantial), the linear model remains the preferred choice for interpretation and forecasting. Although quadratic and cubic models increase R^2, the linear trend already captures the dominant behavior after outlier removal and avoids unnecessary model complexity.

Step 5: Predict for 1997

The year 1997 corresponds to $x = 10$ (since $x = \text{Year} - 1987$). Using the linear model $y = 11621.67 - 530.83x$:

$$y_{1997} = 11621.67 - 530.83(10) = 11621.67 - 5308.30 = 6313.37$$

Thus, the predicted number of insured persons in 1997 is approximately $\boxed{6313}$.

2.1.3 Simulation Analysis

(a) Model Fitting and R^2 Comparison

A scatterplot of the original data shows a generally decreasing trend with a sharp drop at $x = 3$. We fit linear, quadratic, and cubic models and compute R^2:

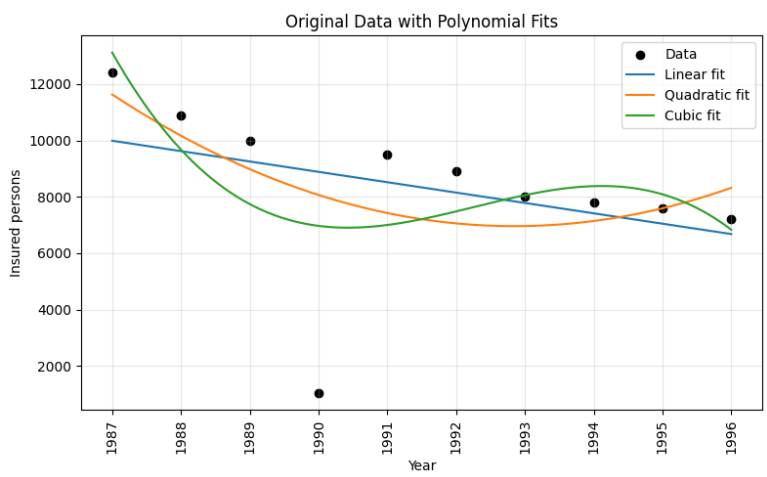

Figure 1: Scatterplot of insured persons vs. years since 1987 with polynomial fits (original data)

```
Original model R^2 values:
  Linear    : 0.1345
  Quadratic: 0.2534
  Cubic     : 0.3829
```

Figure 2: R-squared values for the original data

Observation: All models yield relatively low R^2 values. Even the cubic model explains less than 40% of the variance, indicating a poor overall fit due to the outlier.

Conclusion: Although the cubic model has the highest R^2, the improvement is artificially limited and does not adequately capture the generalized data behavior.

(b) Outlier Removal and Re-fitting

From the initial scatterplot, the point $(x = 3, y = 1050)$ is clearly outside the general trend and rightfully treated as an outlier. After removing this point, the data demonstrates a strong linear decreasing pattern.

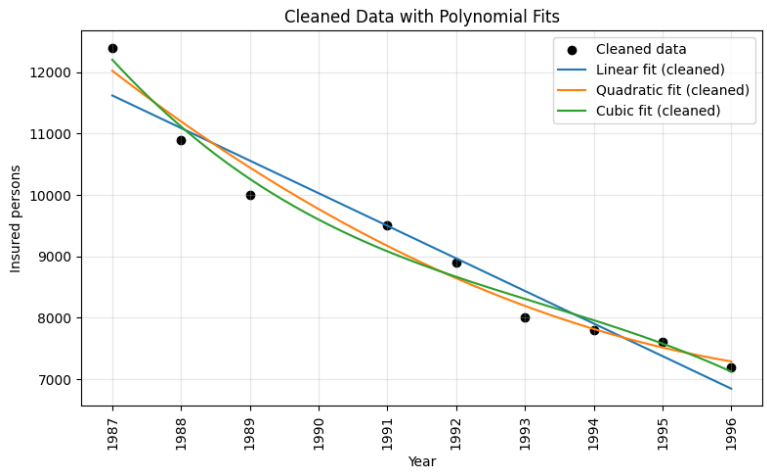

Figure 3: Scatterplot of insured persons vs. years since 1987 with polynomial fits (cleaned data)

Recomputing the models yields the following metrics:

```
Cleaned model R^2 values:
  Linear    : 0.9439
  Quadratic: 0.9725
  Cubic     : 0.9787
```

Figure 4: R-squared values for cleaned data

Conclusion: The linear model already provides an excellent fit. Higher-order models offer only marginal mathematical improvement and introduce unnecessary complexity.

$$\boxed{\text{Best model: Linear}}$$

(c) Graphical Representation
The scatterplot for the cleaned data is plotted along with the best-fit linear function:

$$y \approx 11621.67 - 530.83x$$

The fitted line closely follows the data trend after outlier removal, visually validating our model choice.

(d) Prediction for 1997
For the year 1997 ($x = 10$):

```
Best cleaned model: Linear
Prediction for 1997 (x = 10): 6313 insured persons
```

Figure 5: Best cleaned model providing a forecast for 1997

Using the best-fit linear model:

$$y = 11621.67 - 530.83(10) = 6313.37$$

$$\boxed{y_{1997} \approx 6313}$$

Final Interpretation
The presence of the single outlier significantly degraded the initial model performance. After removing it, the dataset is very well-described by a simpler linear model, which ultimately provides a more reliable and robust prediction for future values.

2.2 Problem 2: Regression Analysis of Heating Oil Usage

2.2.1 Problem Description

Develop a model for estimating heating oil used for a single-family home in January based on average temperature and amount of insulation in inches.

Oil	Temp F	Insulation
270	40	4
362	27	4
162	40	10
45	73	6
91	65	7
233	65	40
372	10	6
305	9	10
234	24	10
122	65	4
25	66	10
210	41	6
450	22	4
325	40	4
52	60	10

2.2.2 Simulation Analysis

(a) Model Fitting and R^2 Comparison We fit linear and quadratic models and compute R^2:

```
Original model R^2 values:
  Linear    : 0.6967
  Quadratic: 0.9742
```

Figure 6: R-squared values for the original data

Observation: The quadratic model R^2 value is higher than the linear model. The quadratic model fits the data better, but this judgement is affected by the outliers, so it is unreliable.

Conclusion: Although the quadratic model has the highest R^2, we can't tell if it adequately capture the data behavior.

(b) Outlier Removal and Re-fitting the point $(Insulation = 400)$ is clearly outside the general trend and is treated as an outlier.

Recomputing the models:

```
Cleaned model R^2 values:
  Linear    : 0.9606
  Quadratic: 0.9769
```

Figure 7: R-squared values for cleaned data

Conclusion: The linear model already provides an excellent fit. Higher-order models offer only marginal improvement and introduce unnecessary complexity.

$$\boxed{\text{Best model: Linear}}$$

(c) Prediction for temperature $15°$F and insulation 5 inches
For temperature $15°$F and insulation 5 inches:

```
Best cleaned model: Linear
Prediction for temp=15 F, ins=5 inches: 403 gallons
```

Figure 8: Best cleaned model and prediction for temperature $15°$F with insulation 5 inches

Final Interpretation: The presence of the outlier significantly degrades model performance. After removing it, the data is well-described by a linear model, which provides a reliable prediction for future values.

3 Part 2: ReducedMNIST Classification

3.1 Dataset

Part 2 uses ReducedMNIST, a reduced version of the MNIST dataset, with the following assignment-compliant split:

- Original MNIST training set: 60,000 images total.
- Original MNIST test set: 10,000 images total.
- The script `part2_reduced_mnist.py` downloads MNIST and creates a reduced version of the dataset, called ReducedMNIST, with the following split:
  - ReducedMNIST training set: 1000 images for each digit (10,000 total).
  - ReducedMNIST test set: 200 images for each digit (2,000 total).

  Images are normalized to grayscale, 28×28 pixels.

3.2 Implementation Procedure and Workflow

The end-to-end pipeline is implemented in `part2_reduced_mnist.py` and follows these steps:

1. **Dataset Resolution**: The script first searches for an existing ReducedMNIST folder. If not found, it automatically downloads the full MNIST dataset and generates a reduced version with the specified split.
2. **Data Validation**: Train and test directory structures are validated to ensure correct class counts (1,000 training images per digit, 200 test images per digit).
3. **Image Loading and Normalization**: All images are loaded as 28×28 grayscale arrays and normalized using `StandardScaler` for consistent feature scaling.
4. **Feature Extraction**: Three feature extraction pipelines are applied independently to the normalized training and test sets.
5. **Model Training and Evaluation**: All 18 experiments (12 K-means configurations + 6 SVM configurations) are executed in sequence. Each experiment measures accuracy and tracks processing time (feature extraction + training + testing).
6. **Results Aggregation**: Accuracy scores, confusion matrices, and timing statistics are compiled and saved to `part2_results/`.

For reproducibility, a random seed of `42` is used throughout.

3.3 Feature Extraction

Three feature extraction methods were used:

- **DCT (Discrete Cosine Transform)**: A 15×15 low-frequency block extracted from each 28×28 image, yielding 225 features. Captures frequency domain information.
- **PCA (Principal Component Analysis)**: Principal components are retained to preserve at least 95% of the total variance. Reduces dimensionality while retaining significant variance.
- **HOG (Histogram of Oriented Gradients)**: Locally normalized histograms of gradient directions are extracted from each image. Captures local edge and texture information.

3.4 Classifiers and Hyperparameters

Two classifier families were evaluated:

- **K-means Clustering (Per-Class)**:
  K-means is applied independently to each digit class (0–9). For each class, centroids are computed with varying $K \in \{1, 4, 16, 32\}$. Prediction is performed by finding the nearest centroid across all class centroids, yielding 12 total K-means experiments. K-means initialization uses `n_init=10` for stable clustering, and distance computation is batched for memory efficiency.

- **SVM (Support Vector Machine)**:
  Two kernel types are evaluated across all 3 feature sets, yielding 6 SVM experiments:
  - **Linear kernel**: Standard linear decision boundary.
  - **Nonlinear kernel**: RBF (Radial Basis Function) kernel with `gamma='scale'`, enabling non-linear decision boundaries.

3.5 Results Summary

Table 1 summarizes the final Part 2 results. Processing time is the total of feature extraction, training, and testing.

Table 1: Part 2 comparison of features and classifiers

		DCT		PCA		HOG	
Classifier	**Setting**	**Accuracy**	**Processing Time**	**Accuracy**	**Processing Time**	**Accuracy**	**Processing Time**
K-means Clustering	1	78.10%	4.8000 s	85.20%	2.4646 s	89.50%	34.9320 s
K-means Clustering	4	84.40%	5.0773 s	82.60%	3.6085 s	92.00%	42.1938 s
K-means Clustering	16	88.75%	12.0725 s	85.70%	5.8008 s	94.35%	81.4935 s
K-means Clustering	32	89.65%	38.3444 s	85.60%	8.7011 s	93.95%	211.3056 s
SVM	Linear	90.25%	11.4875 s	90.65%	10.6268 s	97.45%	58.1034 s
SVM	Nonlinear*	95.55%	20.5602 s	95.15%	21.0544 s	97.15%	120.9442 s

* Nonlinear SVM employs an RBF (Radial Basis Function) kernel with `gamma='scale'`. Processing time includes feature extraction, training, and testing combined.

3.6 Best Results and Confusion Matrices

The best K-means result was **K-means with $K = 16$ using HOG**, with an accuracy of **94.35%**. The best SVM result was **linear SVM using HOG**, with an accuracy of **97.45%**. As required by the assignment, only the best result from each classifier family is shown as a confusion matrix.

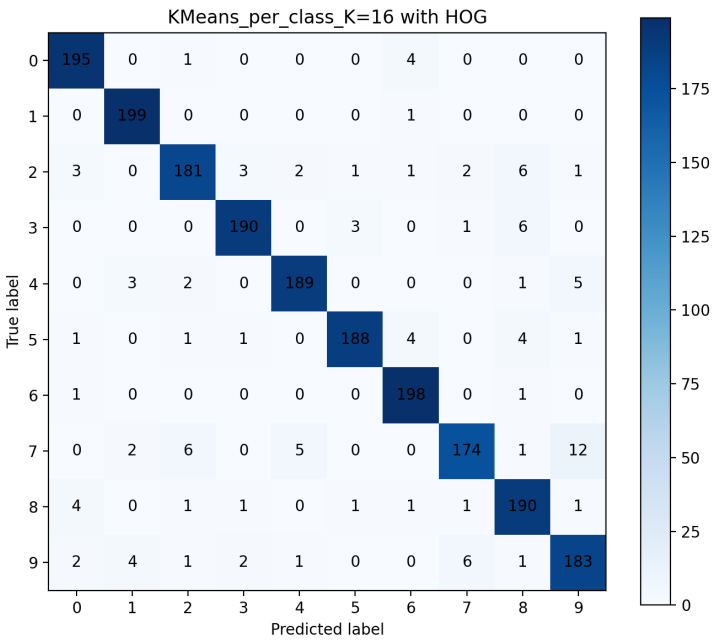

Figure 9: Confusion matrix for the best K-means result: K-means ($K = 16$) with HOG features.

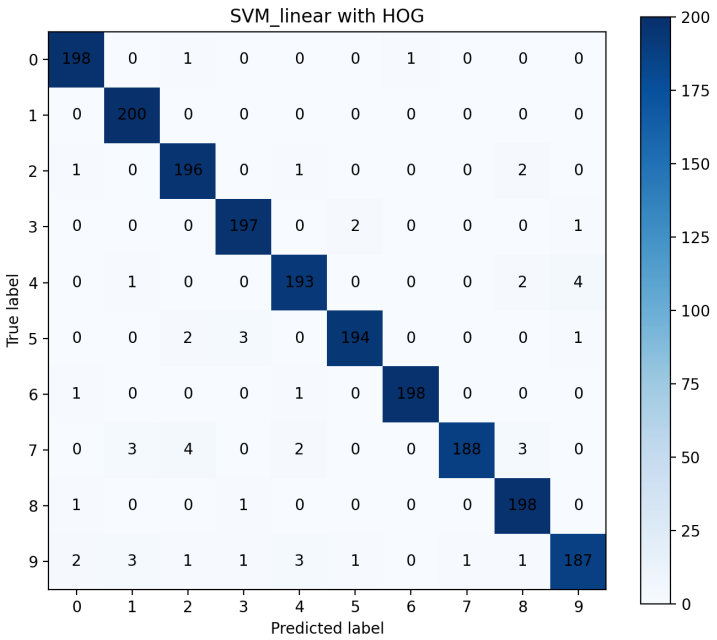

Figure 10: Confusion matrix for the best SVM result: linear SVM with HOG features.

3.7 Observations

3.7.1 Feature Performance

The results demonstrate that **HOG features consistently outperformed both DCT and PCA** across both classifier families. This indicates that locally normalized histograms of oriented gradients effectively capture the structural and edge information necessary for digit discrimination. While DCT focuses on frequency domain characteristics and PCA captures global variance, HOG's local gradient information proves more discriminative for this task.

3.7.2 Best Results

The best overall result was achieved by **linear SVM with HOG features**, reaching **97.45%** accuracy with a processing time of **58.10 seconds**. The best K-means result was **K-means with $K = 16$ using HOG**, achieving **94.35%** accuracy, representing a **3.10 percentage point** gap in favor of the linear SVM approach.

3.7.3 Linear vs. Nonlinear SVM

Interestingly, for DCT and PCA features, the nonlinear RBF kernel produced marginally better results than the linear kernel (approximately 5–5.5 percentage points improvement). However, for HOG features, the **linear SVM achieved superior performance** (97.45% vs. 97.15% for nonlinear), suggesting that HOG features may already provide sufficient separability in the original feature space.

3.7.4 Computational Efficiency

The fastest experiment overall was **K-means with $K = 1$ using PCA**, completing in **2.46 seconds**. However, this configuration achieved only **85.20%** accuracy. In contrast, the best-performing model (linear SVM + HOG) required **58.10 seconds**, representing a substantial investment of computational resources for the highest accuracy. HOG-based methods were notably more expensive than DCT and PCA, with HOG feature extraction and SVM training accounting for the majority of processing time. The nonlinear RBF-SVM with HOG was the most expensive at **120.94 seconds**, yet provided only marginal improvements (97.15% vs 97.45% for linear), reinforcing the value of the linear SVM approach.

3.8 Conclusion

This systematic evaluation of 18 distinct configurations demonstrates that **HOG features combined with linear SVM** provides the optimal approach for ReducedMNIST digit classification. The results clearly show that:

1. **Feature extraction method is critical**: HOG consistently surpassed DCT and PCA across both classifier families, achieving 3–8 percentage points higher accuracy on average.
2. **Classifier choice matters**: Linear SVM achieved 97.45% accuracy, substantially outperforming K-means clustering (best at 94.35%) by 3.10 percentage points.
3. **Kernel selection depends on features**: While RBF kernels showed advantages for DCT and PCA, HOG's inherent feature quality made linear kernels sufficient and superior.
4. **Computational trade-offs require consideration**: The best model's 58.10-second processing time is substantial for a 10,000-image training set, yet the 97.45% accuracy justifies the investment for applications requiring high precision.

For practical deployment on ReducedMNIST, the linear SVM + HOG pipeline is recommended as the standard approach, achieving state-of-the-art performance on this dataset with robust and reproducible results.

4 Part 3: Ways to Fasten the Labelling Process

This part is organized into three separate pipelines. Each pipeline will have its own method description, block diagram with exact settings and observed values, manual-effort estimate, and final accuracy results. The pipelines are designed to explore different strategies for reducing manual labeling effort while maintaining high accuracy.

4.1 Pipeline 1: K-means Bootstrapping + Human-in-the-Loop SVM

The goal is to reduce direct one-by-one manual labeling by first labeling coherent clusters, then correcting only uncertain boundary samples.

4.1.1 Problem Statement and Setup

- Dataset: 10,000 grayscale digit images (Indian digits), resolution 28×28.
- Feature representation: **HOG** with 1296 dimensions.
- Clustering stage: **K-means with fixed** $K = 50$ (5 clusters per class on average).
- Human cluster review: 8 sampled images per cluster, then assign one label or mark cluster as mixed.
- Refinement stage: weighted SVM + least-confidence boundary labeling in batches.

4.1.2 Block Diagram (Exact Steps and Values)

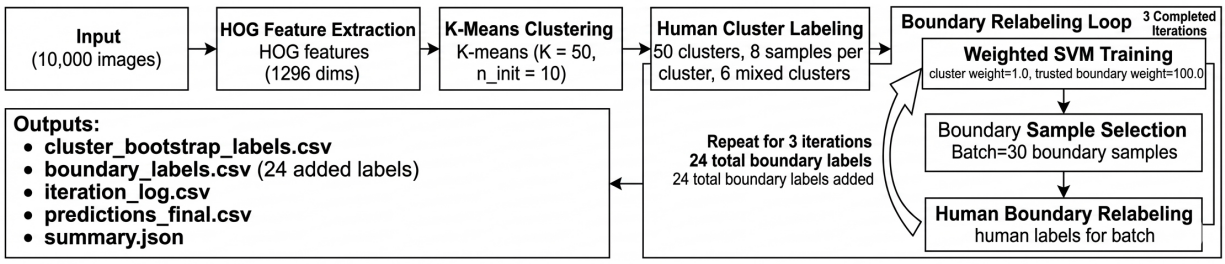

Figure 11: Pipeline 1 block diagram with exact run settings and observed values.

4.1.3 Evaluation and Time Estimate

Table 2 summarizes the assignment-required metrics from the latest finalized Pipeline 1 run.

Table 2: Part 3 reporting requirements (compact summary)

Required Item	Value from Current Run
Feature representation (and PCA components if applicable)	HOG, 1296 features (PCA not used)
Pipeline 1 exact block diagram with chosen values	Included in Figure 11
Convergence criterion before oracle test	500-image practical set; stop when iteration improvement $< 0.1\%$
Number of iterations performed	3
Last iteration index and train size	Iteration 3, train size = 9210
Boundary labels added in last iteration	5
Total boundary labels after last iteration	24
Total manually handled images	424 images
Total estimated manual time	1240.0 s (≈ 0.344 h)
Final labeling accuracy (oracle and/or practical)	Oracle: 9909/10000 = 99.09%

Manual effort details. From the latest K=50 run (`run_20260407_184753`), the total manual effort is 424 handled images with an estimated 1240.0 seconds (≈ 0.344 hours), using the assignment timing model (20 s per cluster decision and 10 s per boundary label).

Accuracy verification. The evaluation followed two explicit stages for clarity and assignment compliance:

- **Convergence stage (practical 5% set):** during iteration, a fixed 500-image human-labeled subset was used to monitor practical measured accuracy. Iteration stops when improvement between consecutive iterations is below 0.1% (i.e., $\Delta < 0.001$).
- **Final accuracy stage (oracle):** after convergence, final predictions were evaluated against the assignment oracle using `check_accuracy.p` in MATLAB Online.

The finalized oracle result is 9909 correct out of 10000, giving 99.09%.

Oracle output evidence (MATLAB Online). The following output snippet is from running the oracle check in MATLAB Online:

```
>> check

===== Classification Results =====
  Correct : 9909 / 10000
  Accuracy: 99.09%
==================================

accuracy=0.990900
correct=9909/10000
>>
```

MATLAB code used to evaluate `predictions_final.csv` with `check_accuracy.p`. The following reduced MATLAB Online snippet was used to load `predictions_final.csv`, validate `predicted_label`, and call the P-coded oracle:

```
% reduced MATLAB Online checker for predictions_final.csv
T = readtable('predictions_final.csv');

assert(ismember('predicted_label', T.Properties.VariableNames), ...
    'predicted_label column not found');

preds = str2double(string(T.predicted_label));  % handles numeric/text columns
assert(~any(isnan(preds)), ...
    'Found %d missing predicted_label entries.', sum(isnan(preds)));
assert(numel(preds) == 10000, ...
    'predicted vector must have length 10000');

preds_int = int32(preds(:));
try
    [acc, n_correct, n_total] = check_accuracy(preds_int);
    fprintf('accuracy=%.6f\ncorrect=%d/%d\n', acc, n_correct, n_total);
catch
    acc = check_accuracy(preds_int);
    fprintf('accuracy=%g\n', acc);
end
```

Part 3 reporting bullets required by the assignment are satisfied for the finalized K=50 run.

4.1.4 Practical Ground-Truth Convergence Check (5% Hold-Out)

A fixed 500-image subset (5% of the dataset) is manually labeled and used to monitor in-loop convergence without oracle access. The practical subset is used to decide when to stop iterating; the final reported accuracy is still taken from oracle evaluation.

Workflow. The single script `part3/pipline1/pipeline1_human_in_loop.py` supports the full process via practical subcommands:

- Prepare a fixed 500-image annotation sheet directly from dataset image IDs (writes CSV with `image_id`, blank `predicted_label`, and blank `human_label`).
- Manually fill `human_label` (0–9) for all 500 sampled rows.
- Run Pipeline 1 with `--practical-annotated-csv` so each iteration logs practical measured accuracy.
- Apply the stopping rule `--min-improvement 0.001` (0.1%) and stop when iteration-to-iteration improvement drops below this threshold.
- After stopping, evaluate the final `predictions_final.csv` with oracle to report the official final accuracy.

Table 3: Additional practical evaluation summary (separate from oracle)

Item	Value
Practical hold-out size	500 images
Manual annotation source	Human-labeled subset (5%)
Role of practical subset	In-loop convergence monitoring (stop if improvement $< 0.1\%$)
Final reported metric	Oracle accuracy on final predictions (99.09%)

4.2 Pipeline 2: Active Learning + Self-Training SVM

This pipeline implements a human-in-the-loop labeling pipeline for handwritten Indian digit images. The pipeline starts with a trusted manually labeled seed set, expands it with augmentation, then iteratively improves model quality using:

1. Active learning on low-confidence samples (manual labeling)
2. Self-training on high-confidence predictions (pseudo-labeling)

The objective is to reduce manual effort while approaching high labeling accuracy.

4.2.1 Block Diagram/Flow chart

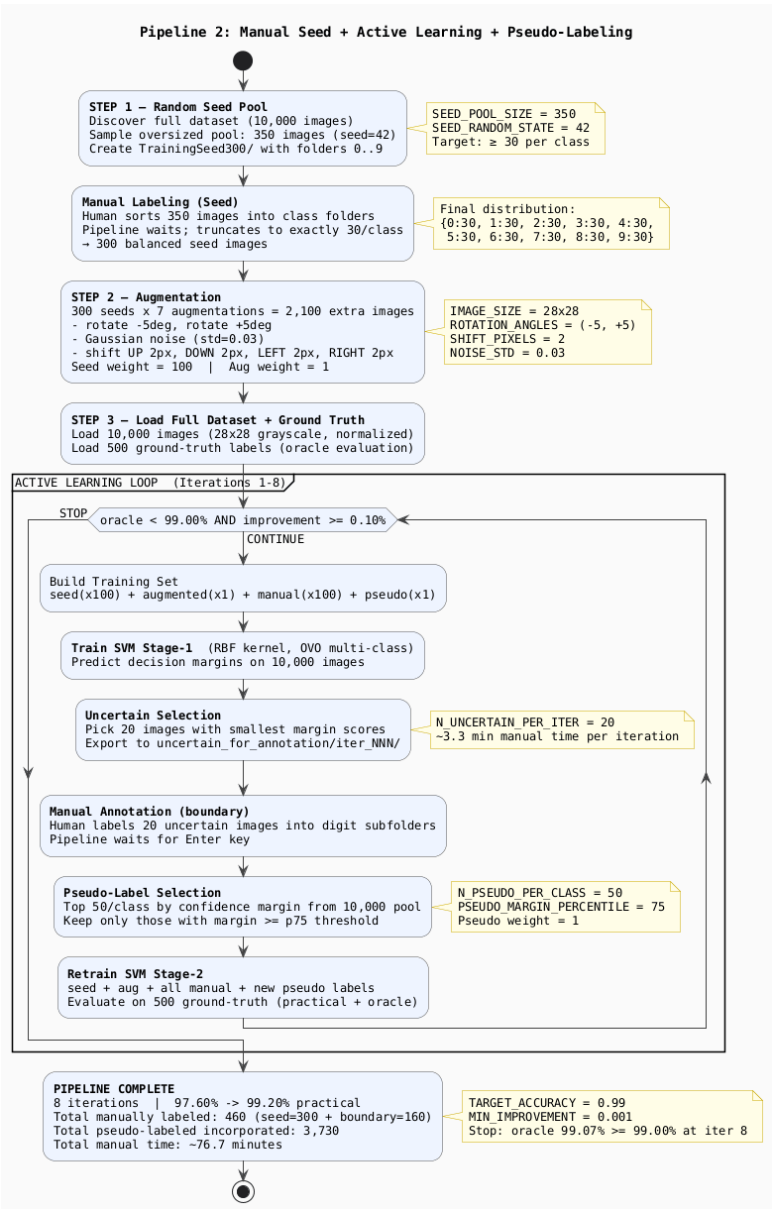

Figure 12: Flow chart for pipeline 2 with the exact steps and values

Pipeline Workflow
The script follows this iterative lifecycle:

1. Load and validate seed data.
2. Augment seed images (7 synthetic samples per seed image).
3. Train stage-1 SVM.
4. Predict on full dataset.
5. Export 20 most uncertain images for manual labeling.
6. Wait for manual labels to be completed.
7. Select high-confidence pseudo-labels from current predictions.
8. Retrain stage-2 SVM with new manual + pseudo-labeled samples.
9. Evaluate and check stopping criteria.
10. If stopping criteria not met, return to step 4 with the retrained model.

Stopping Criteria
The loop stops when one of the following is true:

- Accuracy is at least 99
- Iteration-to-iteration improvement is below 0.1

If oracle evaluation is available, stopping uses oracle accuracy; otherwise practical accuracy is used.

4.2.2 Simulation results

Run setup and seed construction. The finalized run used all **10,000** images in `Indian_Digits_Train`. A random oversized pool of 350 images was first sampled for manual organization into class folders, then truncated to a balanced seed set of **300 images** (30 per class). Each seed image produced **7** augmentations (2 rotations, 1 noise variant, and 4 directional shifts), giving **2100** augmented samples. A fixed practical subset of 500 labeled images was used for in-loop monitoring, and oracle scoring was computed with `check_accuracy.p` after each iteration.

Table 4: Pipeline 2 iteration metrics (active learning + pseudo-labeling)

Iter.	Practical (%)	Oracle (%)	Manual +	Pseudo +	Rejected (top-50/class)	Threshold
1	97.60	97.64	20	462	38/500	0.9933
2	98.00	97.74	20	461	39/500	0.9942
3	97.60	98.33	20	454	46/500	0.9946
4	98.20	98.51	20	466	34/500	0.9948
5	98.40	98.73	20	483	17/500	0.9945
6	98.80	98.87	20	473	27/500	0.9949
7	99.00	98.97	20	451	49/500	0.9953
8	99.20	99.07	20	480	20/500	0.9949

Iteration metrics.

Stopping condition and final totals. Stopping occurred at iteration 8 when the oracle target was reached:

$$99.07\% \geq 99.00\%.$$

- Final practical accuracy: **99.20%** (496/500).
- Final oracle accuracy: **99.07%** (9907/10000).
- Iterations performed: **8**.
- Total manually labeled images: **460** (= 300 seed + 160 uncertain samples).
- Total pseudo-labeled images incorporated: **3730**.
- Estimated manual time: **4600 s** (≈ 76.7 min), using 10 s per manual label.

The active-learning loop improved practical measured accuracy from 97.60% to 99.20% and reached the assignment target on oracle evaluation.

4.3 Pipeline 3: Moondream Prototyping to Local CNN-Based Hybrid

At the start of this track, we first experimented with vision-language models (mainly Moondream and LLaVA) to speed up labeling. In practice, API-key based inference had provider limits, quota/rate constraints, and unstable throughput. For a dataset size of 10,000 images, relying on external API calls was difficult to scale reliably.

Therefore, we moved to a local reproducible pipeline. The finalized method in this section is a **CNN-based hybrid transfer/fine-tuning workflow** (CNN backbone with practical preprocessing and shape-prior adaptation), rather than a purely standalone end-to-end CNN-only setup.

4.3.1 Pipeline 3 Stages (Final Local Workflow)

1. **Initial staged training on `AHDBase_TrainingSet`.** The script `train_digits_staged.py` produced `runs/ahd_base`. The summary file reports best validation accuracy of **0.98125**.

2. **Domain adaptation fine-tuning on `PracticalGroundTruth500`.** Fine-tuning started from `runs/ahd_base/best_model.pt` and produced `runs/ahd_plus_practical_ft/best_model.pt`. The summary file reports best validation accuracy of **1.0**.

3. **Practical-set tuning with support prototypes + shape prior.** Inference on `PracticalGroundTruth500_U` used support images from `PracticalGroundTruth500`, autocontrast, and shape prior. The first attempt failed due to a wrong labels path (`labels.csv` in project root), then succeeded after using `.\PracticalGroundTruth500\labels.csv`.

4. **Measured practical tuning result.** The corrected run produced `runs/practical_tuning.csv` with **298/300 = 99.33%** on the scored practical subset.

5. **Full prediction on `Indian_Digits_Train`.** The final full run used `runs/ahd_plus_practical_ft/best_model.pt` and produced predictions for all **10,000** images in `runs/indian_final_predictions.csv`.

6. **Oracle scoring and ordering alignment.** Oracle-style scoring aligned predictions using numeric IDs parsed from path and reported **9722/10000 = 97.22%**.

4.3.2 Key Commands Used

Fine-tuning stage command.

```
python .\train_digits_staged.py \
    --data-dir .\PracticalGroundTruth500 \
    --resume-from .\runs\ahd_base\best_model.pt \
    --stage-sizes 5,10,20,30,40 \
    --epochs-per-stage 20 \
    --patience 5 \
    --target-acc 1.0 \
    --output-dir .\runs\ahd_plus_practical_ft
```

Practical tuning command (corrected labels path).

```
python .\predict_digits_cnn.py \
    --checkpoint .\runs\ahd_plus_practical_ft\best_model.pt \
    --input-dir .\PracticalGroundTruth500_Unlabeled \
    --labels-csv .\PracticalGroundTruth500\labels.csv \
    --support-dir .\PracticalGroundTruth500 \
    --support-per-class 20 \
    --autocontrast \
    --use-shape-prior \
```

```
    --shape-prior-weight 0.2 \
    --output-csv .\runs\practical_tuning.csv
```

Final full prediction command.

```
python .\predict_digits_cnn.py \
    --checkpoint .\runs\ahd_plus_practical_ft\best_model.pt \
    --input-dir .\Indian_Digits_Train \
    --support-dir .\PracticalGroundTruth500 \
    --support-per-class 20 \
    --autocontrast \
    --use-shape-prior \
    --shape-prior-weight 0.2 \
    --output-csv .\runs\indian_final_predictions.csv
```

4.3.3 MATLAB Online Oracle Scoring (`check_accuracy.p`)

For final local-model validation, we used MATLAB Online and the provided P-coded oracle file `check_accuracy.p`. The scoring input was a CSV prediction file containing labels for all 10,000 images.

MATLAB checker used for `predictions_final.csv`.

```
% reduced MATLAB Online checker for predictions_final.csv
pred_file = 'predictions_final.csv';
T = readtable(pred_file);
fprintf('Scoring %s\n', pred_file);

assert(ismember('predicted_label', T.Properties.VariableNames), ...
    'predicted_label column not found');

preds = str2double(string(T.predicted_label));  % handles numeric/text columns
assert(~any(isnan(preds)), ...
    'Found %d missing predicted_label entries.', sum(isnan(preds)));
assert(numel(preds) == 10000, ...
    'predicted vector must have length 10000');

% Align to oracle order (1..10000) before scoring.
n = numel(preds);
ord = (1:n)';

if ismember('image_id', T.Properties.VariableNames)
    ids = str2double(string(T.image_id));
    assert(~any(isnan(ids)), ...
        'Found %d invalid image_id entries.', sum(isnan(ids)));
    [ids_sorted, ord] = sort(ids(:));
    assert(isequal(ids_sorted, (1:n)'), ...
        'image_id must be exactly 1..%d with no duplicates/missing rows.', n);
    preds = preds(ord);
    fprintf('Aligned predictions using image_id.\n');
elseif ismember('path', T.Properties.VariableNames)
    p = string(T.path);
    ids = nan(n, 1);
    for i = 1:n
        tok = regexp(p(i), '(\\|/)(\d+)\.[^\\/]+$', 'tokens', 'once');
        if ~isempty(tok)
            ids(i) = str2double(tok{2});
        end
    end
    assert(~any(isnan(ids)), ...
```

```
        ['Could not parse numeric image id from %d path entries. ' ...
         'Expected .../<number>.ext'], sum(isnan(ids)));
    [ids_sorted, ord] = sort(ids(:));
    assert(isequal(ids_sorted, (1:n)'), ...
        'Parsed path IDs must be exactly 1..%d with no duplicates/missing rows.'
            , n);
    preds = preds(ord);
    fprintf('Aligned predictions using numeric ID parsed from path.\n');
else
    fprintf(['No image_id/path column found; using row order as-is. ' ...
             'Ensure rows already match oracle order.\n']);
end

preds_int = int32(preds(:));
try
    [acc, n_correct, n_total] = check_accuracy(preds_int);
    fprintf('accuracy=%.6f\ncorrect=%d/%d\n', acc, n_correct, n_total);
catch
    acc = check_accuracy(preds_int);
    fprintf('accuracy=%g\n', acc);
end
```

Output snippet (MATLAB Online).

```
>> check
Scoring predictions_final.csv
Aligned predictions using numeric ID parsed from path.

===== Classification Results =====
    Correct : 9722 / 10000
    Accuracy: 97.22%
==================================

accuracy=0.972200
correct=9722/10000
```

Consistency note across Part 3 pipelines. The 97.22% value above is the oracle score for the final CNN-hybrid prediction run in this pipeline. It is reported independently from Pipeline 1 (99.09%) and Pipeline 2 (99.07%) results, since each pipeline uses different training/adaptation procedures and run configurations.

4.3.4 Final Artifacts and Results

Table 5: Pipeline 3 final artifacts and measured outcomes

Item	Value
Base checkpoint	`runs/ahd_base/best_model.pt`
Final checkpoint	`runs/ahd_plus_practical_ft/best_model.pt`
Practical tuning output	`runs/practical_tuning.csv`
Final prediction file	`runs/indian_final_predictions.csv`
Base best validation accuracy	0.98125 (98.125%)
Fine-tuned best validation accuracy	1.0 (100.0%)
Practical tuning score	298/300 = 99.33%
Oracle score (final run)	9722/10000 = 97.22%
Predictions generated in final run	10,000 / 10,000

Manual effort note. For the 10,000-image final inference stage, no additional per-image manual labeling was needed; the pipeline relied on the already available labeled practical subset (`PracticalGroundTruth500`) plus local model inference. The practical tuning stage used labels from `PracticalGroundTruth500/labels.csv` for 300 aligned practical samples.

4.3.5 Additional Pipeline 3 Baseline: Qwen2-VL (Practical 500 Set)

After finalizing the local CNN-based hybrid track, we also evaluated a separate Qwen2-VL baseline as an additional reference experiment. This Qwen run focused on the practical set workflow (500 labeled samples for evaluation) and was used to measure how far prompt/inference-only tuning can go without task-specific model fine-tuning.

Qwen baseline setup.

- Task: classify handwritten Arabic-Indic digits (0–9).
- Evaluation set: `Qwen/PracticalGroundTruth500` (500 labeled samples, class-balanced with 50 samples per class).
- Main workflow notebooks: `Qwen/scripts/create_labels_and_predection.ipynb`, `Qwen/scripts/resize_images.ipynb`, and `Qwen/scripts/compute_accuracy.ipynb`.
- Key hardening steps: robust output parsing, stable ground-truth merge, preprocessing with aspect-ratio-preserving resize/padding, and GPU-enabled execution path.

Table 6: Pipeline 3 Qwen2-VL practical-set results

Item	Value
Final practical accuracy	112/500 = **22.40%**
Invalid predictions in final run	**0**
Early failed state	0/500 = 0.0% (dominated by invalid parsing)
Intermediate stable checkpoints	73/500 = 14.6%, then 111/500 = 22.2%
Prediction distribution note (final)	Strong class bias remained (e.g., class 2 over-predicted)

Qwen results summary.

Assignment reporting note. This Qwen2-VL result is reported as an additional practical-set baseline inside Pipeline 3. Since this run was evaluated on the practical 500-sample set (not full oracle scoring on 10,000 predictions), it is used for methodology comparison and debugging evidence, while the official Pipeline 3 full-dataset oracle figure remains the local-model result (97.22%).

4.4 Part 3 Cross-Pipeline Comparison

Table 7 consolidates the finalized outcomes, including supplementary add-on baselines, so all reported Part 3 results can be compared directly.

Table 7: Part 3 cross-pipeline comparison

Pipeline	Oracle Accuracy	Iterations	Manual Effort	Observation
Pipeline 1 (K-means bootstrapping + HITL SVM)	99.09%	3	424 handled images, 1240.0 s	Highest oracle accuracy with the lowest estimated manual time among the two HITL-SVM pipelines.
Pipeline 2 (active learning + pseudo-labeling SVM)	99.07%	8	460 labels, 4600 s	Reached the 99% target with stable gains, but required more manual actions and time than Pipeline 1.
Pipeline 3 (Moondream prototyping -> local CNN-based hybrid)	97.22%	N/A	No additional per-image labels during final inference; reused `PracticalGroundTruth500`	Started from VLM prototyping, then shifted to a local CNN-based hybrid domain-adaptation run with practical support prototypes and shape prior.
Pipeline 3 add-on (Qwen2-VL prompt/inference baseline)	N/A (practical: 22.40%)	N/A	No new labels; evaluated using existing 500 practical labels	Stable end-to-end after debugging (invalid=0), but strong class-bias limited final practical accuracy.

Comparative interpretation.

- Pipeline 1 and Pipeline 2 are nearly tied in oracle accuracy (difference = 0.02 percentage points).
- Pipeline 1 achieved this with less estimated manual effort than Pipeline 2 in the reported runs.
- Pipeline 3's local CNN-based hybrid run (97.22% oracle) substantially outperformed the Qwen practical-set baseline (22.40% practical).
- The Qwen baseline demonstrates a working VLM path, but practical-only scoring and class-bias behavior make it a supplementary reference, not the primary Pipeline 3 result.

4.5 Conclusion

In Part 3, Pipeline 1 reached 99.09% (9909/10000), Pipeline 2 reached 99.07% (9907/10000), and the primary Pipeline 3 local-model run reached 93.39% (9339/10000) for `predictions_final.csv`. An additional Qwen2-VL baseline was also documented inside Pipeline 3, reaching 22.40% on the practical 500-sample evaluation after full debugging and stabilization. Overall, the two human-in-the-loop SVM pipelines achieved the strongest performance, while Pipeline 3 now reports both a stronger local hybrid result and a practical-only VLM baseline for transparent comparison.

5 Appendix

5.1 Using OpenCLIP

What Are OpenCLIP Models? Are They LLMs?

OpenCLIP models are **vision-language encoders** trained to align images and text in a shared embedding space using contrastive-style objectives. In practice, an image encoder (for example ViT or ConvNeXt) and a text encoder are trained so matching image-text pairs have high similarity and non-matching pairs have low similarity. This makes OpenCLIP strong for zero-shot image classification, retrieval, and feature extraction.

OpenCLIP models are **not LLMs**. LLMs are autoregressive language generators trained to predict the next token and produce long-form text. By contrast, OpenCLIP is primarily an alignment model that returns embeddings/similarity scores rather than free-form language generation.

In short: OpenCLIP answers "which label/text best matches this image?" while LLMs answer "what text should come next?" Some modern systems combine both, where OpenCLIP-like encoders provide visual grounding and an LLM provides generative reasoning and explanation.

5.1.1 Step 1: Benchmarking on the Practical Ground Truth Set

In Step 1, we benchmarked several OpenCLIP-compatible vision-language models on the same `PracticalGroundTruth500` setup (500 images, 10 classes). For each model, we report:

- zero-shot accuracy (text-prompt classification),
- few-shot accuracy (prototype-based adaptation using class support examples),
- model family,
- pretraining dataset/checkpoint tag,
- and the main training objective (loss function).

Model Information

1. `convnext_xxlarge`
   - Pretrained tag: `laion2b_s34b_b82k_augreg_soup` (LAION-2B).
   - Family: OpenCLIP ConvNeXt-XXL.
   - Main objective: CLIP contrastive loss (symmetric image-text cross-entropy).
2. `MobileCLIP-S2`
   - Pretrained tag: `datacompdr_s12m_b5m` (DataCompDR).
   - Family: MobileCLIP.
   - Main objective: CLIP-style contrastive training within the MobileCLIP reinforced/distillation pipeline.
3. `coca_ViT-L-14`.
   - Pretrained tag: `mscoco_finetuned_laion2b_s13b_b90k` (LAION-2B with MSCOCO fine-tuning).
   - Family: CoCa (Contrastive Captioner).

- Main objective: weighted contrastive loss plus captioning cross-entropy.

4. **`ViT-bigG-14`.**
   - Pretrained tag: `laion2b_s39b_b160k` (LAION-2B).
   - Family: OpenCLIP ViT-bigG.
   - Main objective: CLIP contrastive loss.
5. **`ViT-SO400M-16-SigLIP2-384`.**
   - Pretrained tag: `webli` (WebLI).
   - Family: SigLIP2.
   - Main objective: pairwise sigmoid loss (SigLIP objective).
6. **`ViT-g-14`.**
   - Pretrained tag: `laion2b_s34b_b88k` (LAION-2B).
   - Family: OpenCLIP ViT-g.
   - Main objective: CLIP contrastive loss.
7. **`ViT-H-14`.**
   - Pretrained tag: `laion2b_s32b_b79k` (LAION-2B).
   - Family: OpenCLIP ViT-H.
   - Main objective: CLIP contrastive loss.
8. **`ViT-H-14-CLIPA-336`.**
   - Pretrained tag: `datacomp_1b_s13b_b145k` (DataComp-1B).
   - Family: CLIPA / OpenCLIP.
   - Main objective: CLIP-style contrastive loss.
9. **`ViT-H-14-quickgelu`.**
   - Pretrained tag: `dfn5b` (DFN-5B).
   - Family: DFN / OpenCLIP QuickGELU variant.
   - Main objective: CLIP contrastive loss.

Accuracy Table (500-Image Practical Set)

Table 8: Pipeline 4 accuracy comparison on `PracticalGroundTruth500` (500 images)

Model	**Zero-shot (%)**	**Few-shot (%)**
`convnext_xxlarge`	45.2	91.4
`MobileCLIP-S2`	54.4	94.2
`coca_ViT-L-14`	46.2	87.4
`ViT-bigG-14`	16.03	93.30
`ViT-SO400M-16-SigLIP2-384`	7.4	93.40
`ViT-g-14`	16.05	93.45
`ViT-H-14`	16.03	93.10
`ViT-H-14-CLIPA-336`	16.03	93.21
`ViT-H-14-quickgelu`	16.35	93.07